

Applied Type Theory

Лекция 5: ADT и алгебра типов

Воронов Михаил Сергеевич

ВМК МГУ

Осень 2025

План лекции

- 1 Мотивация и основные понятия
- 2 Паттерн-матчинг, произведения и суммы
- 3 Алгебра типов
- 4 Рекурсивные ADT и полиномиальные функторы
- 5 Классы типов (ad-hoc полиморфизм)
- 6 Практические рекомендации
- 7 Практикум: QUIC протокол как ADT (quinn-proto)
- 8 Заключение

- 1 **Мотивация и основные понятия**
- 2 Паттерн-матчинг, произведения и суммы
- 3 Алгебра типов
- 4 Рекурсивные ADT и полиномиальные функторы
- 5 Классы типов (ad-hoc полиморфизм)
- 6 Практические рекомендации
- 7 Практикум: QUIC протокол как ADT (quinn-proto)
- 8 Заключение

Зачем расширять STLC?

В чистом исчислении $\lambda_{\rightarrow}$ есть только функции. Но в реальных программах нужны:

- **Пары и кортежи** — для комбинирования значений: (x, y) , координаты, пары ключ-значение
- **Варианты и суммы** — для альтернатив: `Either`, `Option`, обработка ошибок
- **Константы** — для тривиальных случаев: `unit ()`, пустой тип для невозможных ситуаций

Алгебраические типы данных (ADT) (неформально) — типы, построенные из произведений ($\times$) и сумм ($+$).

Примеры из языков:

- Haskell: `data Either a b = Left a | Right b`
- OCaml: `type ('a, 'b) either = Left of 'a | Right of 'b`
- Rust: `enum Result<T, E> { Ok(T), Err(E) }`

Formation / Introduction / Elimination / Computation

Каждый тип в теориях типов удобно вводить через шаблон F/I/E/C:

Formation (формирование)

Если σ и τ — типы, то $\sigma \rightarrow \tau$ — тип.

Introduction (введение)

Абстракция: если $\Gamma, x:\sigma \vdash M : \tau$, то $\Gamma \vdash \lambda x. M : \sigma \rightarrow \tau$.

Elimination (элиминация)

Аппликация: если $\Gamma \vdash F : \sigma \rightarrow \tau$ и $\Gamma \vdash N : \sigma$, то $\Gamma \vdash F N : \tau$.

Computation (вычисления)

Дефиниционные равенства для функций (β , η):

$$(\lambda x. M) N \rightarrow_{\beta} M[x := N] \qquad \lambda x. F x =_{\eta} F \quad (x \notin \text{FV}(F))$$

Почему именно F/I/E/C паттерн

Обоснование подхода

- **Модульность спецификации:** отдельно фиксируем, что является типом (Formation), как строятся его значения (Introduction), как их использовать (Elimination) и какие равенства считать очевидными (Computation).
- **Каноничность и исполняемость:** интродукция задаёт канонические формы, а элиминация с вычислениями описывает их применение, обеспечивая прогресс и сохранение типов.
- **Универсальные свойства:** элиминаторы и β/η -равенства выражают универсальные свойства $\times$ и $+$ (единственность морфизмов), что делает законы ожидаемыми и достаточными.
- **Практика языков:** эти четыре аспекта дают полный и предсказуемый API типов: конструкторы, деструкторы/паттерн-матчинг и эквивалентности «из коробки».

План лекции

- 1 Мотивация и основные понятия
- 2 Паттерн-матчинг, произведения и суммы
- 3 Алгебра типов
- 4 Рекурсивные ADT и полиномиальные функторы
- 5 Классы типов (ad-hoc полиморфизм)
- 6 Практические рекомендации
- 7 Практикум: QUIC протокол как ADT (quinn-proto)
- 8 Заключение

Произведение ($\times$): правила и законы

Formation & Introduction

$$\frac{\sigma \text{ тип} \quad \tau \text{ тип}}{\sigma \times \tau \text{ тип}}$$

$$\frac{\Gamma \vdash M : \sigma \quad \Gamma \vdash N : \tau}{\Gamma \vdash (M, N) : \sigma \times \tau} \times I$$

Elimination

$$\frac{\Gamma \vdash P : \sigma \times \tau}{\Gamma \vdash \pi_1 P : \sigma} \pi_1 \quad \frac{\Gamma \vdash P : \sigma \times \tau}{\Gamma \vdash \pi_2 P : \tau} \pi_2$$

Computation (β)

$$\pi_1(M, N) \equiv M \quad \pi_2(M, N) \equiv N$$

Computation (η)

$$(\pi_1 p, \pi_2 p) \equiv p$$

Операционная семантика

$$\pi_1(M, N) \rightarrow M \quad \pi_2(M, N) \rightarrow N$$

Сумма (+): правила и законы

Formation & Introduction

$$\frac{\sigma \text{ тип} \quad \tau \text{ тип}}{\sigma + \tau \text{ тип}}$$

$$\frac{\Gamma \vdash M : \sigma}{\Gamma \vdash \text{inl } M : \sigma + \tau} +I_l \quad \frac{\Gamma \vdash N : \tau}{\Gamma \vdash \text{inr } N : \sigma + \tau} +I_r$$

Elimination

$$\frac{\Gamma \vdash L : \sigma + \tau \quad \Gamma, x:\sigma \vdash M : \rho \quad \Gamma, y:\tau \vdash N : \rho}{\Gamma \vdash \text{case } L \text{ of inl } x \rightarrow M \mid \text{inr } y \rightarrow N : \rho} +E$$

Computation (β)

case (inl M) of inl $x \rightarrow M' \mid \text{inr } y \rightarrow N' \equiv M'[x := M]$

case (inr N) of inl $x \rightarrow M' \mid \text{inr } y \rightarrow N' \equiv N'[y := N]$

Computation (η)

case L of inl $x \rightarrow \text{inl } x \mid \text{inr } y \rightarrow \text{inr } y \equiv L$

Unit (1) и Void (0)

Unit (1) — единичный тип

$$\frac{}{1 \text{ тип}} \quad \frac{}{\Gamma \vdash () : 1} 1I$$

Elimination

$$\frac{\Gamma \vdash M : 1 \quad \Gamma, x:1 \vdash N : \tau}{\Gamma \vdash N : \tau} 1E$$

где практически всегда N не зависит от x (так как x не несёт информации).

η -закон: любой терм типа 1 равен $()$.

Интуиция: аналог $()$ в Haskell/OCaml и $()$ в Rust; тип с единственным значением, не несущий информации. Не путать с `void` в C (который обозначает отсутствие возвращаемого значения, но не является типом с единственным жителем).

Void (0) — пустой тип

$$\frac{}{0 \text{ тип}}$$

Нет интродукции (нет конструкторов).

Элиминация: *ex falso quodlibet*

$$\frac{\Gamma \vdash M : 0}{\Gamma \vdash \text{absurd } M : \tau} 0E$$

Интуиция: тип без значений; из противоречия следует что угодно (*ex falso quodlibet*). Если мы имеем значение типа 0, мы можем получить значение *любого* типа τ — это не проблема, так как получить значение типа 0 невозможно. Аналоги: `never`-тип `!` в Rust; `Nothing` в Scala.

Match как элиминация; проекции как деструкторы

Match = элиминация суммы

$$\frac{\Gamma \vdash L : \sigma + \tau \quad \Gamma, x:\sigma \vdash M : \rho \quad \Gamma, y:\tau \vdash N : \rho}{\Gamma \vdash \text{case } L \text{ of } \text{inl } x \rightarrow M \mid \text{inr } y \rightarrow N : \rho} +E$$

Синтаксический сахар: `match` $\equiv$ `case`.

Деструкторы для $\times$

$$\pi_1 : \sigma \times \tau \rightarrow \sigma, \quad \pi_2 : \sigma \times \tau \rightarrow \tau$$

$$\text{let } (x,y)=p \text{ in } M \equiv M[x := \pi_1 p, y := \pi_2 p].$$

- Вложенные шаблоны транслируются в композиции π_i и вложенные case.
- Пример: `match L with Inl (x,y) -> M | Inr z -> N`

$$\text{case } L \text{ of inl } p \rightarrow M[x := \pi_1 p, y := \pi_2 p] \mid \text{inr } z \rightarrow N$$

- **Покрытие вариантов (exhaustiveness):** требуем, чтобы все ветки суммы были обработаны.
- **Статическая проверка:** компилятор выдаёт warning/error о неохваченных случаях.
- **Связь с safety:** исчерпываемость + сохранение типов $\Rightarrow$ отсутствие «stuck» при разборе.

Примеры (Haskell/OCaml)

Haskell

```
eitherLen :: Eq b => Either a (b,b) -> Int
eitherLen e = case e of
  Left _      -> 1
  Right (x,y) -> if x==y then 2 else 3
```

OCaml

```
type ('a,'b) either =
  Left of 'a | Right of 'b

let either_len = function
  | Left _ -> 1
  | Right (x,y) -> if x = y then 2 else 3
```

Учебный `eitherLen` пример демонстрирует:

- Паттерн-матчинг по конструкторам `Left/Right`
- Вложенный паттерн (пара внутри `Right`)
- Условие на деструктурированных значениях

Упражнение: переписать `match` с вложенными парами через чистые $+E$ и π_i ; упростить по η .

Алгебраические типы данных (ADT): определение

Интуитивно

Алгебраический тип данных (ADT) — тип, получаемый конечными композициями сумм ($+$), произведений ($\times$) и константных типов ($0, 1$), возможно с параметрами.

Формально (на уровне сигнатур)

Если σ, τ — типы, то $\sigma + \tau$, $\sigma \times \tau$, 0 , 1 — тоже типы; ADT — любой тип, выражаемый этой грамматикой. Конструкторы интродукции — это *варианты* для суммы и *построение пары* для произведения; элиминация — *pattern matching*/проекции.

Примеры:

- $\text{Either } A \ B \cong A + B$, $(A, B) \cong A \times B$
- $\text{Option } A \cong 1 + A$
- $\text{Bool} \cong 1 + 1$

План лекции

- 1 Мотивация и основные понятия
- 2 Паттерн-матчинг, произведения и суммы
- 3 Алгебра типов**
- 4 Рекурсивные ADT и полиномиальные функторы
- 5 Классы типов (ad-hoc полиморфизм)
- 6 Практические рекомендации
- 7 Практикум: QUIC протокол как ADT (quinn-proto)
- 8 Заключение

Жители типа (inhabitants)

Количество различных значений данного типа.

- $|0| = 0$ — невозможно создать значение типа 0
- $|1| = 1$ — ровно одно значение (unit)
- $|\text{Bool}| = 2$ — два значения: True, False
- $|\sigma + \tau| = |\sigma| + |\tau|$ — сумма вариантов
- $|\sigma \times \tau| = |\sigma| \cdot |\tau|$ — произведение (все комбинации пар)
- $|\sigma \rightarrow \tau| = |\tau|^{|\sigma|}$ — число всех возможных функций

Пример: $|\text{Bool} \rightarrow \text{Bool}| = 2^2 = 4$
(id, not, const True, const False)

Что такое алгебра типов

Идея

Рассматриваем типы как элементы алгебры с операциями $+$, $\times$, $\rightarrow$ и константами 0 , 1 . Законы этой алгебры выражаются через *изоморфизмы типов*.

Напоминание: полукольцо

Полукольцо — это структура $(S, +, \cdot, 0, 1)$ такая, что:

- $(S, +, 0)$ — коммутативный моноид: $+$ ассоциативно и коммутативно, 0 — нейтральный элемент для $+$;
- $(S, \cdot, 1)$ — моноид: $\cdot$ ассоциативно, 1 — нейтральный элемент для $\cdot$;
- $\cdot$ дистрибутивно относительно $+$: $a \cdot (b + c) = a \cdot b + a \cdot c$ и $(a + b) \cdot c = a \cdot c + b \cdot c$;
- 0 — аннулирующий элемент (поглощающий): $0 \cdot a = a \cdot 0 = 0$.

Экстенсиональная интерпретация

Мощности множеств: $|\sigma + \tau| = |\sigma| + |\tau|$, $|\sigma \times \tau| = |\sigma| \cdot |\tau|$, $|\sigma \rightarrow \tau| = |\tau|^{|\sigma|}$; отсюда следуют коммутативность, ассоциативность, дистрибутивность и нейтральные элементы.

Вывод

С операциями суммы $+$, произведения $\times$ и константами $0, 1$ алгебра типов образует *полукольцо* (на уровне изоморфизмов типов). Стрелка $\rightarrow$ играет роль экспоненциальной операции.

Польза: позволяет выводить и запоминать изоморфизмы, понимать структуру ADT и проектировать API по законам F/I/E/C.

Алгебраическая интерпретация

Типы образуют алгебру с операциями $\times$, $+$, $\rightarrow$ и константами 0, 1.

Алгебраическая интерпретация

- $\times$ — произведение: $|\sigma \times \tau| = |\sigma| \cdot |\tau|$ (число пар)
- $+$ — сумма: $|\sigma + \tau| = |\sigma| + |\tau|$ (число вариантов)
- $\rightarrow$ — экспонента: $|\sigma \rightarrow \tau| = |\tau|^{|\sigma|}$ (число функций)
- 1 — единица: $|1| = 1$
- 0 — ноль: $|0| = 0$

Соответствие типов и арифметики

Haskell	Математика	Примечания
<code>data Void</code>	0	Невозможно создать терм этого типа (пустой тип без конструкторов)
<code>data Unit = Unit</code>	1	Тип с единственным обитателем
<code>data Bool = True False</code>	$1 + 1 \cong 2$	Изоморфно 2 (аналогично определяются 3, 4 и т.д.)
<code>data Maybe a = Just a Nothing</code>	$a + 1$	«+» - «или»
<code>data Either a b = Left a Right b</code>	$a + b$	
<code>(a, b)</code>	$a \times b$	«×» — «и»
<code>a -> b</code>	b^a	тип функций из a в b

Примеры изоморфизмов

- $\sigma \times \tau \cong \tau \times \sigma$ (коммутативность произведения)
- $\sigma + \tau \cong \tau + \sigma$ (коммутативность суммы)
- $\sigma \times (\tau + \rho) \cong (\sigma \times \tau) + (\sigma \times \rho)$ (дистрибутивность)
- $\sigma \times 1 \cong \sigma, \quad \sigma + 0 \cong \sigma$ (нейтральные элементы)
- $\sigma^{(\tau+\rho)} \cong \sigma^\tau \times \sigma^\rho, \quad (\sigma^\rho)^\tau \cong \sigma^{(\rho \times \tau)}$ (законы экспонент)

Алгебраические манипуляции: практический пример

Рассмотрим тип: `data Choice a = LeftChoice a | RightChoice a`

Алгебраическая запись: $a + a$

Применим школьную алгебру:

$$a + a = 2 \times a = \text{Bool} \times a$$

Получаем изоморфный тип: `type Choice' a = (Bool, a)`

Интуиция:

- `Choice` помечает каждое значение как «левое» или «правое»
- `Choice'` использует булев флаг для той же цели
- Алгебраические преобразования согласуются с нашей интуицией!

Упражнение: Объяснить, почему $\text{Bool} \rightarrow a$ изоморфно (a, a) .

Подсказка: функция $f : \text{Bool} \rightarrow a$ полностью определяется парой значений $(f(\text{True}), f(\text{False}))$. Обратно, из пары $(x, y) : (a, a)$ строим функцию $g(b) = \text{if } b \text{ then } x \text{ else } y$. Это взаимно обратные изоморфизмы.

Формальные ряды для списков

Рассмотрим списки: $L = 1 + a \times L$

Развернём определение итеративно:

$$\begin{aligned} L &= 1 + a \times L \\ &= 1 + a \times (1 + a \times L) \\ &= 1 + a + a^2 \times L \\ &= 1 + a + a^2 + a^3 + \dots \end{aligned}$$

Интерпретация: список — это либо пустой список, либо список из одного элемента, либо из двух, и т.д.

Используя алгебру формальных степенных рядов (геометрическая серия для $(1 - a)^{-1}$):

$$L = 1 + a \times L \quad \Rightarrow \quad L(1 - a) = 1 \quad \Rightarrow \quad L = \frac{1}{1 - a} = \sum_{i=0}^{\infty} a^i$$

Замечание: алгебраические манипуляции корректны в кольце формальных степенных рядов.

Дифференцирование типов

Идея

Производная типа $\frac{\partial T}{\partial a}$ — это тип T с «дыркой» вместо одного вхождения a .

Правила дифференцирования

- $\frac{\partial}{\partial a} c = 0$ (константа не содержит a)
- $\frac{\partial}{\partial a} a = 1$ (одна дырка типа a)
- $\frac{\partial}{\partial a} (f + g) = \frac{\partial f}{\partial a} + \frac{\partial g}{\partial a}$
- $\frac{\partial}{\partial a} (f \times g) = \frac{\partial f}{\partial a} \times g + f \times \frac{\partial g}{\partial a}$

Пример: $\frac{\partial}{\partial a} (a \times a) = 1 \times a + a \times 1 = 2a$

Пара элементов с дыркой в одной из двух позиций.

Zipper: производная как контекст

Zipper

Структура данных для эффективного «фокуса» на одном элементе с возможностью перемещения влево/вправо за $O(1)$.

Для списка $L = 1 + a \times L$:

$$\frac{\partial L}{\partial a} = 0 + 1 \times L + a \times \frac{\partial L}{\partial a} = L + a \times \frac{\partial L}{\partial a}$$

Решая уравнение (учитывая $L = \frac{1}{1-a}$ из предыдущего слайда):

$$\frac{\partial L}{\partial a} = L + a \times \frac{\partial L}{\partial a} \Rightarrow \frac{\partial L}{\partial a}(1 - a) = L \Rightarrow \frac{\partial L}{\partial a} = \frac{L}{1 - a} = L \cdot L = L^2$$

Интерпретация: L^2 — это два списка: левый (развёрнутый) и правый.

Вывод: Дифференцирование структуры данных даёт тип её контекста, который можно использовать как zipper!

Пример: Zipper для списка

Список с фокусом на элементе 3:

[]	<-	:	<-	:	<-	FOCUS	->	:	->	[]
		1		2		3		4		

Тип: $([a], a, [a]) \cong L \times a \times L$, где $L \times L = L^2$, согласуется с $\frac{\partial L}{\partial a} = L^2$

Операции за $O(1)$:

- Переместить фокус влево/вправо
- Изменить сфокусированный элемент
- Вставить/удалить элемент

Обычное редактирование произвольного элемента списка — $O(n)$,
с zipper — $O(1)$ для каждой операции.

Почему это работает?

ADT образуют *полукольцо* — алгебраическую структуру с операциями $+$ и $\times$.

- Все алгебраические манипуляции обоснованы теорией категорий
- Изоморфизмы типов соответствуют изоморфизмам в категории типов
- Дифференцирование типов впервые замечено Конором МакБрайдом (2001)
- Связь с исчислением открывает новые возможности для проектирования структур данных

Дополнительное чтение:

- Joel Burget, “The algebra (and calculus!) of algebraic data types”
<https://codewords.recurse.com/issues/three/algebra-and-calculus-of-algebraic-data-types>
- Conor McBride, “The Derivative of a Regular Type is its Type of One-Hole Contexts”

План лекции

- 1 Мотивация и основные понятия
- 2 Паттерн-матчинг, произведения и суммы
- 3 Алгебра типов
- 4 Рекурсивные ADT и полиномиальные функторы**
- 5 Классы типов (ad-hoc полиморфизм)
- 6 Практические рекомендации
- 7 Практикум: QUIC протокол как ADT (quinn-proto)
- 8 Заключение

Полиномиальные функторы

Определение

Полиномиальный функтор по типовой переменной X имеет вид

$$F(X) = \sum_i a_i \times X^{k_i}$$

где a_i — фиксированные типы коэффициентов, $k_i \in \mathbb{N}$, $X^0 = 1$, $X^{k+1} = X \times X^k$.

Интуиция

Суммы соответствуют альтернативам (конструкторам), произведения — полям конструктора, степени X^k — числу рекурсивных подузлов.

Рекурсивные типы как фиксированные точки

Мю-нотация

Рекурсивный тип задаётся как наименьшая фиксированная точка: $\mu X. F(X)$, где F — полиномиальный функтор.

Примеры

- Списки: $\text{List } A \cong \mu X. (1 + A \times X)$.
- Полные бинарные деревья: $\mu X. (1 + A \times X \times X)$.
- Натуральные числа Пеано: $\mu X. (1 + X)$.

Подстановка: $\text{List } A \cong 1 + A \times \text{List } A$ (итеративная подстановка, развёртывание фиксированной точки).

Замечание: для непротиворечивой семантики требуется *строгая положительность* (strict positivity) X в F (т.е. X не встречается слева от стрелки в определении F , что гарантирует существование начальной алгебры).

Катаморфизмы: мотивация

Проблема: многие рекурсивные функции на списках выглядят похоже

Сумма элементов

$$\begin{aligned}\text{sum } [] &= 0 \\ \text{sum } (x : xs) &= x + \text{sum } xs\end{aligned}$$

Конкатенация

$$\begin{aligned}\text{concat } [] &= [] \\ \text{concat } (xs : xss) &= xs ++ \text{concat } xss\end{aligned}$$

Длина списка

$$\begin{aligned}\text{length } [] &= 0 \\ \text{length } (x : xs) &= 1 + \text{length } xs\end{aligned}$$

Map

$$\begin{aligned}\text{map } f [] &= [] \\ \text{map } f (x : xs) &= f x : \text{map } f xs\end{aligned}$$

Общий паттерн

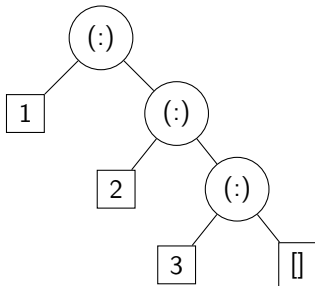
Все функции:

- Обработывают [] константой (или константной функцией)
- Обработывают (x : xs) комбинацией x и рекурсивного результата

Идея: выделить этот паттерн в обобщённую функцию!

Интуиция: замена конструкторов

Ключевая идея: список — это «дерево» из конструкторов $[]$ и $(:)$



Список $[1, 2, 3] = 1 : (2 : (3 : []))$

Катаморфизм (fold) = «замена конструкторов»

$\text{foldr } f \ z$ заменяет:

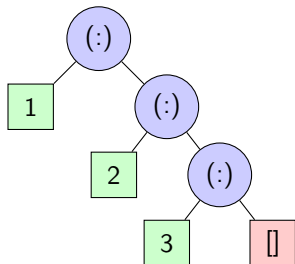
- каждый $(:)$ на функцию f
- каждый $[]$ на значение z

Пример:

$\text{sum } [1, 2, 3] = \text{foldr } (+) \ 0 \ [1, 2, 3]$
 $= 1 + (2 + (3 + 0)) = 6$

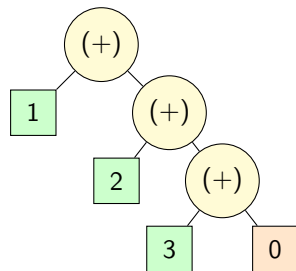
Визуализация: как работает foldr

Исходный список



$\text{foldr } (+) 0$

Результат вычисления



Наблюдение: структура не меняется, меняются только «метки» узлов!

От конкретного к общему: foldr

Определение foldr:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z []      = z
foldr f z (x:xs) = f x (foldr f z xs)
```

Интуиция:

- f — функция для обработки элемента и накопленного результата
- z — начальное значение (для пустого списка)
- $\text{foldr } f \ z \ [x_1, x_2, \dots, x_n] = f \ x_1 \ (f \ x_2 \ (\dots (f \ x_n \ z) \dots))$

Ключевая идея

foldr заменяет каждый конструктор $(:)$ на функцию f , а $[]$ на значение z

Примеры выражения через foldr

Огромное количество функций на списках — это просто foldr с разными аргументами!

```
sum = foldr (+) 0
```

```
length = foldr (\_ n -> 1 + n) 0
```

```
product = foldr (*) 1
```

```
concat = foldr (++) []
```

```
map g = foldr (\x xs -> g x : xs) []
```

```
filter p = foldr (\x xs ->  
    if p x then x:xs else xs) []
```

Вывод

Все эти функции следуют одному паттерну: они *сворачивают* список, применяя операцию к каждому элементу и накопленному результату.

Обобщение: катаморфизмы для произвольных типов

Вопрос: а можем ли мы сделать то же самое для деревьев? Для других рекурсивных типов?

Ответ: да, это и есть *катаморфизмы*.

Катаморфизм = обобщённый fold

Для *любого* рекурсивного типа $T = \mu X. F(X)$ можно определить универсальную операцию «свёртки», которая:

- 1 Обходит всю структуру
- 2 Заменяет каждый конструктор на заданную операцию
- 3 Вычисляет результат снизу вверх (от листьев к корню)

Примеры:

- Списки: $\text{List } A = \mu X. (1 + A \times X) \Rightarrow \text{foldr}$
- Деревья: $\text{Tree } A = \mu X. (1 + A \times X \times X) \Rightarrow \text{foldTree}$
- Натуральные числа: $\text{Nat} = \mu X. (1 + X) \Rightarrow \text{итерация}$

Формализация: что нужно для катаморфизма?

Чтобы определить катаморфизм для типа $T = \mu X. F(X)$, нужно задать:

1. Функтор F

Описывает «форму одного шага» рекурсии.

Пример: для списков $F(X) = 1 + A \times X$ означает «либо пусто, либо элемент + хвост».

2. Алгебра $\alpha : F(A) \rightarrow A$

Описывает, как «схлопнуть» один уровень структуры в результат.

Пример: для `sum` алгебра — это (z, f) , где:

- $z : 1 \rightarrow \mathbb{N}$ задаёт результат для пустого списка: $z() = 0$
- $f : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ задаёт комбинацию: $f(x, acc) = x + acc$

Катаморфизм — это функция, которая применяет α ко всем уровням рекурсии

F-алгебры: формальное определение (упрощённо)

Что такое F-алгебра?

Пусть F — функтор (например, $F(X) = 1 + A \times X$).

F -алгебра — это пара:

- Тип A (называемый *носителем* или *carrier*)
- Функция $\alpha : F(A) \rightarrow A$ (как «схлопнуть» структуру)

Примеры F-алгебр для списков ($F(X) = 1 + \mathbb{N} \times X$):

- 1 $A = \mathbb{N}$, $\alpha(inl()) = 0$, $\alpha(inr(n, acc)) = n + acc$ (это *sum*)
- 2 $A = \mathbb{N}$, $\alpha(inl()) = 0$, $\alpha(inr(n, acc)) = 1 + acc$ (это *length*)
- 3 $A = [\mathbb{N}]$, $\alpha(inl()) = []$, $\alpha(inr(n, acc)) = n : acc$ (это *id*)

Интуиция: алгебра = «рецепт обработки одного слоя структуры».

Начальная алгебра и катаморфизм

Начальная алгебра

Сам рекурсивный тип $T = \mu X. F(X)$ с конструктором $\text{in} : F(T) \rightarrow T$ образует *начальную* F-алгебру.

«Начальная» означает: это «самая общая» алгебра, из которой есть уникальный путь к любой другой.

Катаморфизм = универсальная стрелка

Для любой F-алгебры (A, α) существует *единственная* функция

$$\text{cata } \alpha : T \rightarrow A$$

которая «согласована» с конструкторами.

На практике: катаморфизм — это способ определить функцию $T \rightarrow A$, просто указав, что делать с каждым конструктором!

Магия: единственность гарантирует, что определение корректно и не зависит от того, как мы представляем структуру.

Универсальное свойство катаморфизмов

Катаморфизм $\text{cata } \alpha : T \rightarrow A$ удовлетворяет коммутативной диаграмме:

$$\begin{array}{ccc} T & \xrightarrow{\text{cata } \alpha} & A \\ \text{in} \uparrow & & \uparrow \alpha \\ F(T) & \xrightarrow{F(\text{cata } \alpha)} & F(A) \end{array}$$

Уравнение катаморфизма

$$\text{cata } \alpha \circ \text{in} = \alpha \circ F(\text{cata } \alpha)$$

Читается: «применить конструктор, потом свернуть» = «рекурсивно свернуть части, потом применить алгебру»

Важно: это уравнение *однозначно определяет* $\text{cata } \alpha$!

Пример: катаморфизм для списков

Рассмотрим $\text{List } A = \mu X. (1 + A \times X)$

Функтор

$$F(X) = 1 + A \times X$$

F-алгебра

Алгебра (B, α) задаётся:

- $z : 1 \rightarrow B$ — для $[]$
- $f : A \times B \rightarrow B$ — для $(:)$

Итого: $\alpha : (1 + A \times B) \rightarrow B$

Катаморфизм = foldr

$$\text{cata } (z, f) = \text{foldr } f \ z$$

Вычисление:

- $\text{foldr } f \ z \ [] = z$
- $\text{foldr } f \ z \ (x : xs) = f \ x \ (\text{foldr } f \ z \ xs)$

Связь с универсальным свойством

foldr — это в точности катаморфизм для типа списков: единственная функция, удовлетворяющая уравнению $\text{cata } \alpha \circ \text{in} = \alpha \circ F(\text{cata } \alpha)$.

Примеры катаморфизмов для списков

Сумма элементов:

- Алгебра: $z = 0$, $f = (+)$
- $\text{sum} = \text{foldr } (+) 0$

Длина списка:

- Алгебра: $z = 0$, $f = \lambda x n. 1 + n$
- $\text{length} = \text{foldr } (\lambda_ n. 1 + n) 0$

Конкатенация списка списков:

- Алгебра: $z = []$, $f = (++)$
- $\text{concat} = \text{foldr } (++) []$

Мар через fold:

- Для $g : A \rightarrow B$: алгебра $z = []$, $f = \lambda x xs. (g\ x) : xs$
- $\text{map } g = \text{foldr } (\lambda x xs. (g\ x) : xs) []$

Визуализация: $[1, 2, 3] = 1 : (2 : (3 : []))$

$\text{foldr } f\ z\ [1, 2, 3] = f\ 1\ (f\ 2\ (f\ 3\ z))$ — «замена конструкторов»

Пример: катаморфизм для бинарных деревьев

Рассмотрим $\text{Tree } A = \mu X. (1 + A \times X \times X)$

Функтор

$$F(X) = 1 + A \times X \times X$$

F-алгебра для деревьев

Алгебра (B, α) задаётся:

- $l : 1 \rightarrow B$ — обработчик листа
- $n : A \times B \times B \rightarrow B$ — обработчик узла с двумя поддеревьями

Катаморфизм для дерева

$$\text{foldTree } l \text{ } n \text{ Leaf} = l$$

$$\text{foldTree } l \text{ } n \text{ (Node } a \text{ } t_1 \text{ } t_2) = n \text{ } a \text{ (foldTree } l \text{ } n \text{ } t_1) \text{ (foldTree } l \text{ } n \text{ } t_2)$$

Примеры катаморфизмов для деревьев

Высота дерева:

- Алгебра: $l = 0$, $n = \lambda a \ h_1 \ h_2. 1 + \max(h_1, h_2)$
- `height = foldTree 0 ( $\lambda\_ h_1 \ h_2. 1 + \max(h_1, h_2)$ )`

Подсчёт узлов:

- Алгебра: $l = 0$, $n = \lambda a \ c_1 \ c_2. 1 + c_1 + c_2$
- `countNodes = foldTree 0 ( $\lambda\_ c_1 \ c_2. 1 + c_1 + c_2$ )`

Зеркальное отражение:

- Алгебра: $l = \text{Leaf}$, $n = \lambda a \ t_1 \ t_2. \text{Node } a \ t_2 \ t_1$
- `mirror = foldTree Leaf ( $\lambda a \ t_1 \ t_2. \text{Node } a \ t_2 \ t_1$ )`

Обход in-order (список значений):

- Алгебра: $l = []$, $n = \lambda a \ xs_1 \ xs_2. xs_1 ++ [a] ++ xs_2$

Законы катаморфизмов

Закон отражения (Reflection law)

$$\text{cata in} = \text{id}$$

Катаморфизм, использующий конструктор как алгебру, является тождественной функцией.

Закон слияния (Fusion law)

Если $h \circ \alpha = \beta \circ F(h)$, то:

$$h \circ \text{cata } \alpha = \text{cata } \beta$$

Позволяет оптимизировать композиции катаморфизмов.

Закон банана (Banana split law)

$$\langle \text{cata } \alpha, \text{cata } \beta \rangle = \text{cata } \langle \alpha, \beta \rangle$$

Два катаморфизма можно вычислить за один проход.

Применение: эти законы используются для оптимизации программ и доказательства

Функторность: map для List/Tree

List

$$\text{map}_{\text{List}} : (A \rightarrow B) \rightarrow \text{List } A \rightarrow \text{List } B$$

Законы: $\text{map id} = \text{id}$,
 $\text{map}(g \circ f) = \text{map}(g) \circ \text{map}(f)$.

Tree

$$\text{map}_{\text{Tree}} : (A \rightarrow B) \rightarrow \text{Tree } A \rightarrow \text{Tree } B$$

Определяется индуктивно по структуре (или через F -функторность).

Примеры кода: foldr, sum, height

Haskell

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z xs = case xs of
    []      -> z
    (y:ys) -> f y (foldr f z ys)

sum :: Num a => [a] -> a
sum = foldr (+) 0

data Tree a = Leaf | Node a (Tree a) (Tree a)

height :: Tree a -> Int
height t = case t of
    Leaf -> 0
    Node _ l r -> 1 + max (height l) (height r)
```

OCaml

```
let rec foldr f z xs =
  match xs with
  | [] -> z
  | y::ys -> f y (foldr f z ys)

let sum xs = List.fold_right (+) xs 0

type 'a tree = Leaf | Node of 'a * 'a tree * 'a tree

let rec height t = match t with
  | Leaf -> 0
  | Node (_, l, r) -> 1 + max (height l) (height r)
```

Упражнение 1: натуральные числа

Задача: вывести (и реализовать) обобщённый fold для натуральных чисел Пеано

$$\mathbb{N} = \mu X. (1 + X)$$

где конструкторы: $\text{Zero} : \mathbb{N}$ и $\text{Succ} : \mathbb{N} \rightarrow \mathbb{N}$

Определите подходящую алгебру $\alpha : F(B) \rightarrow B$ и выразите через катаморфизм:

- Сложение с константой: $\text{add } n : \mathbb{N} \rightarrow \mathbb{N}$
- Умножение на константу: $\text{mult } n : \mathbb{N} \rightarrow \mathbb{N}$
- Факториал: $\text{factorial} : \mathbb{N} \rightarrow \mathbb{N}$

Решение: катаморфизм для натуральных чисел

Функтор: $F(X) = 1 + X$

F-алгебра: алгебра (B, α) задаётся двумя компонентами:

- $z : B$ — обработчик нуля
- $s : B \rightarrow B$ — обработчик следующего числа

Катаморфизм:

```
foldNat :: b -> (b -> b) -> Nat -> b
foldNat z s Zero      = z
foldNat z s (Succ n) = s (foldNat z s n)
```

Примеры:

```
add k      = foldNat k Succ
mult k     = foldNat Zero (add k)
factorial = foldNat (Succ Zero) (\n -> mult (Succ n) n)
```

Упражнение 2: арифметические выражения

Задача: вывести (и реализовать) обобщённый fold для арифметических выражений

$$\text{Expr} = \mu X. (\mathbb{N} + X \times X + X \times X)$$

где конструкторы: $\text{Lit} : \mathbb{N} \rightarrow \text{Expr}$, $\text{Add} : \text{Expr} \times \text{Expr} \rightarrow \text{Expr}$, $\text{Mul} : \text{Expr} \times \text{Expr} \rightarrow \text{Expr}$

Определите подходящую алгебру $\alpha : F(B) \rightarrow B$ и выразите через катаморфизм:

- Интерпретатор (вычисление значения)
- Подсчёт количества операций
- Печать выражения в строку

Решение: катаморфизм для выражений

Функтор: $F(X) = \mathbb{N} + X \times X + X \times X$

F-алгебра: алгебра (B, α) задаётся тремя функциями:

- $lit : \mathbb{N} \rightarrow B$ — обработчик литералов
- $add : B \times B \rightarrow B$ — обработчик сложения
- $mul : B \times B \rightarrow B$ — обработчик умножения

Катаморфизм:

```
foldExpr :: (Int -> b) -> (b -> b -> b) -> (b -> b -> b)
           -> Expr -> b
foldExpr lit add mul (Lit n)      = lit n
foldExpr lit add mul (Add e1 e2) =
    add (foldExpr lit add mul e1) (foldExpr lit add mul e2)
foldExpr lit add mul (Mul e1 e2) =
    mul (foldExpr lit add mul e1) (foldExpr lit add mul e2)
```

Решение: примеры использования

1. Интерпретатор (вычисление значения):

```
eval :: Expr -> Int
eval = foldExpr id (+) (*)
```

2. Подсчёт операций:

```
countOps :: Expr -> Int
countOps = foldExpr (const 0)
                (\c1 c2 -> 1 + c1 + c2)
                (\c1 c2 -> 1 + c1 + c2)
```

3. Печать в строку:

```
prettyPrint :: Expr -> String
prettyPrint = foldExpr show
                (\s1 s2 -> "(" ++ s1 ++ " + " ++ s2 ++ ")")
                (\s1 s2 -> "(" ++ s1 ++ " * " ++ s2 ++ ")")
```

Зачем нужны катаморфизмы?

Теоретическая значимость

- **Универсальность:** единый принцип для всех индуктивных типов
- **Корректность:** автоматически следует из универсального свойства начальной алгебры
- **Композициональность:** катаморфизмы композируются и оптимизируются (fusion laws)

Практические применения

- **Компиляторы:** обход AST, оптимизации, кодогенерация
- **Парсеры:** построение результата из разобранных структуры
- **Сериализация:** преобразование структур данных в/из форматов (JSON, XML)
- **Visitor pattern:** катаморфизм — это функциональный аналог Visitor
- **Рекурсивные схемы:** библиотеки recursion-schemes в Haskell

Катаморфизмы: ключевые выводы

- ❶ Катаморфизм — это **универсальный паттерн** для определения функций на индуктивных типах
- ❷ Достаточно указать, **что делать с каждым конструктором** — рекурсия происходит автоматически
- ❸ Катаморфизмы обладают **алгебраическими законами** (fusion, identity), позволяющими проводить формальные доказательства и оптимизации
- ❹ Это не **абстракция ради абстракции**, а практичный инструмент для структурированной работы с рекурсивными данными

Основная идея: «разделяй и властвуй» для рекурсивных типов — описывая локальную логику, получаем глобальное поведение.

План лекции

- 1 Мотивация и основные понятия
- 2 Паттерн-матчинг, произведения и суммы
- 3 Алгебра типов
- 4 Рекурсивные ADT и полиномиальные функторы
- 5 Классы типов (ad-hoc полиморфизм)**
- 6 Практические рекомендации
- 7 Практикум: QUIC протокол как ADT (quinn-proto)
- 8 Заключение

Мотивация: generic serializer для ADT

Задача: хотим сериализовать любой ADT в строку/JSON/bytes.

Проблема:

- Разные типы требуют разных стратегий: $\text{Int} \rightarrow "42"$, $\text{String} \rightarrow "hello"$, $\text{List} \rightarrow "[...]"$
- Хотим единый интерфейс: $\text{serialize} : \forall a. a \rightarrow \text{String}$
- Но параметрический полиморфизм не даёт заглянуть внутрь a (параметричность!)

Что нужно?

- 1 **Комбинирование** результатов: $\text{serialize} (\text{Left } x) = \text{"Left("} + \text{serialize } x + \text{"}"$
- 2 **Маппинг** перед сериализацией: преобразовать элементы, сохранив структуру
- 3 **Обход** контейнеров: списки, деревья, Maybe

Решение: классы типов — словари операций с разными реализациями для разных типов.

Ad-hoc полиморфизм: три подхода

Три вида полиморфизма (классификация Cardelli-Wegner, 1985)

- **Параметрический:** $\forall a. a \rightarrow a$ — одна реализация для всех типов
- **Ad-hoc (перегрузка):** $\forall a. \text{Serialize } a \Rightarrow a \rightarrow \text{String}$ — разные реализации для разных a
- **Subtype:** наследование и динамическая диспетчеризация (ООП)

Класс типов = словарь операций + законы

Структура:

- Набор операций (методов)
- Неформальные законы, которым должны удовлетворять операции
- Множество инстансов (реализаций) для конкретных типов

Примечание: существуют и другие виды полиморфизма (row polymorphism, higher-rank, kind polymorphism), но эти три — основа классификации.

Шаг 1: Semigroup и Monoid для комбинирования

Задача: нужно комбинировать части сериализованных данных.

Пример: `serialize (Left x) = "Left(" ⊕ serialize x ⊕ ")"`

Semigroup — комбинирование

```
class Semigroup a where
  (<>) :: a -> a -> a
  -- Law: (x <> y) <> z = x <> (y <> z)

instance Semigroup String where
  (<>) = (++)

instance Semigroup [a] where
  (<>) = (++)
```

Monoid — с пустым элементом

```
class Semigroup a => Monoid a where
  mempty :: a
  -- Laws: mempty <> x = x = x <> mempty

instance Monoid String where
  mempty = ""

instance Monoid [a] where
  mempty = []
```

Применение: конкатенация результатов сериализации — ассоциативная операция

Шаг 2: Functor для маппинга

Задача: преобразовать элементы внутри контейнера, сохранив структуру.

Пример: `serialize [1,2,3] = "[1, 2, 3]"` — нужно применить `show` к каждому элементу.

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
  -- Laws: fmap id = id,  fmap (g . f) = fmap g . fmap f

instance Functor [] where
  fmap = map

instance Functor (Either e) where
  fmap f (Left e)  = Left e
  fmap f (Right x) = Right (f x)

instance Functor Maybe where
  fmap f Nothing  = Nothing
  fmap f (Just x) = Just (f x)
```

Functor: применение для сериализации

Идея: используем `fmap` для преобразования элементов перед комбинированием.

Для списков:

```
serializeList :: Show a
              => [a] -> String
serializeList xs =
  let strs = fmap show xs
  in "[" <>
    intercalate ", " strs <>
    "]"

-- Example:
-- serializeList [1,2,3]
--   = "[1, 2, 3]"
```

Для Either:

```
-- Transform Right value
serializeEither ::
  (Show a, Show b) =>
  Either a b -> String
serializeEither e =
  case fmap show e of
    Left x  ->
      "{\"Left\": " <> x <> "}"
    Right y ->
      "{\"Right\": " <> y <> "}"
```

Паттерн: `fmap` + комбинирование через `<>` (Monoid).

Шаг 3: Foldable для обхода структур

Задача: обойти все элементы контейнера, собрав результаты сериализации.

Связь с катаморфизмами: Foldable — это обобщение foldr на любые контейнеры!

```
class Foldable t where
  foldr :: (a -> b -> b) -> b -> t a -> b
  -- Law: folding respects structure

instance Foldable [] where
  foldr f z []      = z
  foldr f z (x:xs) = f x (foldr f z xs)

instance Foldable (Either e) where
  foldr f z (Left _)  = z
  foldr f z (Right x) = f x z

instance Foldable Maybe where
  foldr f z Nothing  = z
  foldr f z (Just x) = f x z
```

Применение:

```
-- Serialize a list
serializeList :: Serialize a
              => [a] -> String
serializeList xs =
  "[" <>
  foldr combine "" xs <>
  "]"
where
  combine x acc =
    serialize x <>
    (if null acc
     then ""
     else ", " <> acc)
```

Dictionary-passing: как это работает

Компилятор преобразует классы типов в явную передачу словарей (records).

Исходный код:

```
serializeList :: Serialize a  
              => [a] -> String  
serializeList xs =  
    "[" <> intercalate ", "  
    (fmap serialize xs) <> "]"
```

После desugaring:

```
data SerializeDict a = SD {  
    serialize :: a -> String }  
  
serializeList ::  
    SerializeDict a -> [a] -> String  
serializeList dict xs =  
    "[" <> intercalate ", "  
    (fmap (serialize dict) xs)  
    <> "]"
```

Следствие: после inline словари исчезают → zero-cost abstraction

Законы классов типов и параметричность

Основные законы

- **Semigroup/Monoid:** $(x <> y) <> z = x <> (y <> z)$, $\text{mempty} <> x = x$
- **Functor:** $\text{fmap id} = \text{id}$, $\text{fmap}(g \circ f) = \text{fmap } g \circ \text{fmap } f$
- **Foldable:** обход в предсказуемом порядке, согласованность с моноидальностью

Параметричность и автоматические гарантии

Для полиморфной функции $\text{fmap} : \forall a b. (a \rightarrow b) \rightarrow F a \rightarrow F b$ сам тип ограничивает возможные реализации.

Если реализация полиморфна (без `unsafeCoerce`/рефлексии), **то**:

- Закон композиции часто следует автоматически из параметричности
- Функция не может «заглянуть» внутрь $a \rightarrow$ должна сохранять структуру

Вывод: типы дают *частичные* гарантии законов; полная проверка требует зависимых типов или property-based testing.

Free theorems: теоремы бесплатно из типов

Идея Wadler (1989)

Из полиморфного типа функции автоматически следуют «*бесплатные теоремы*» (free theorems) — утверждения о поведении функции, которые выполняются для *всех* её реализаций.

Пример 1: $\text{reverse} :: [a] \rightarrow [a]$

«Бесплатная теорема»:

$$\forall f : a \rightarrow b. \quad \text{map } f \circ \text{reverse} = \text{reverse} \circ \text{map } f$$

Интуиция: `reverse` не может «заглянуть» внутрь элементов (параметричность), поэтому порядок применения `map` и `reverse` не важен.

Доказательство: следует автоматически из теоремы о параметричности Reynolds (1983).

Free theorems: практические примеры

Пример 2: `head :: [a] -> a`

«Бесплатная теорема»: $\forall f : a \rightarrow b, xs : [a]. \quad f (\text{head } xs) = \text{head } (\text{map } f \text{ } xs)$

Пример 3: `foldr :: (a -> b -> b) -> b -> [a] -> b`

«Бесплатная теорема» (упрощённо):

$$\forall h : b \rightarrow c. \quad h \circ \text{foldr } f \text{ } z = \text{foldr } (\lambda x y. h (f \text{ } x \text{ } y)) (h \text{ } z)$$

если h — гомоморфизм моноида.

Применение

- **Оптимизации:** fusion laws следуют из free theorems
- **Рефакторинг:** безопасные преобразования кода
- **Верификация:** автоматическое доказательство свойств
- **Понимание API:** тип говорит, что функция *не может* делать

Free theorems: когда они нарушаются

Free theorems справедливы только при соблюдении параметричности.

Нарушители параметричности:

- `unsafeCoerce` (Haskell): принудительное приведение типов
- `typeof`, рефлексия (Java, C#): проверка типа в рантайме
- `undefined`, `error`: дивергенция и исключения
- `seq`: строгость и оценка порядка
- FFI и низкоуровневые операции

Вывод: строгая параметричность — необходимое условие для free theorems. В чистых ФП языках (Haskell без расширений, System F) они гарантированы.

План лекции

- 1 Мотивация и основные понятия
- 2 Паттерн-матчинг, произведения и суммы
- 3 Алгебра типов
- 4 Рекурсивные ADT и полиномиальные функторы
- 5 Классы типов (ad-hoc полиморфизм)
- 6 Практические рекомендации**
- 7 Практикум: QUIC протокол как ADT (quinn-proto)
- 8 Заключение

Проектирование API с ADT: ключевые принципы

Основные идеи:

- 1 **Make illegal states unrepresentable** — невалидные состояния непредставимы
- 2 Используйте **суммы** для взаимоисключающих состояний
- 3 Используйте **произведения** для обязательных полей
- 4 **Phantom types** для протоколов (zero-cost)
- 5 **Newtypes** для семантической безопасности
- 6 **Smart constructors** для валидации инвариантов

Сумма vs произведение: практический выбор

Произведение — все поля всегда есть

- Используем `Option` для опциональных
- Много невалидных состояний
- **Плохо**: допускает ошибки

```
data Config = Config String (Maybe Int) Bool
```

Принцип: «Make illegal states unrepresentable» (Yaron Minsky)

Сумма — каждый конструктор = состояние

- Невалидные состояния непредставимы
- Exhaustiveness checking
- **Хорошо**: безопасно по построению

```
data Config = Basic String | Advanced String Int
```

Phantom types: протоколы в типах

Идея: кодируем состояния протокола в параметре типа (zero-cost).

```
// Phantom type states
type opened
type authenticated
type closed

// Connection with phantom state parameter
type connection (s:Type) = | Conn of handle

val open_conn : string -> ML (connection opened)
val authenticate : connection opened -> password ->
    ML (connection authenticated)
val send : connection authenticated -> bytes -> ML unit
val close : #s:Type -> connection s -> ML (connection closed)

// send conn "Hello"  -- TYPE ERROR if conn : connection opened
```

Гарантия: невозможно вызвать send без аутентификации — ошибка компиляции!

Newtypes: семантическая безопасность

Проблема: примитивы не выражают семантику

```
fn transfer(from: String, to: String, amount: f64) { ... }  
transfer(user_name, account_id, 100.0); -- compiler won't help!
```

Решение: newtypes (zero-cost wrappers)

```
struct AccountId(String);  
struct UserName(String);  
struct Amount(f64);  
  
fn transfer(from: AccountId, to: AccountId, amount: Amount) { ... }  
transfer(user_name, account_id, Amount(100.0)); -- TYPE ERROR!
```

Эффект: компилятор проверяет корректность, но wrapper стирается при компиляции.

Smart constructors: валидация в типах

Идея: скрыть raw конструктор, экспортировать только валидатор.

```
module Email

// Abstract type - constructor is private!
abstract type email = | Email of string

// Smart constructor with validation
val mk_email : string -> Tot (option email)
let mk_email s =
  if String.contains s '@' && String.contains s '.'
  then Some (Email s)
  else None

// Getter for accessing the value
val un_email : email -> Tot string
let un_email (Email s) = s

// let bad = Email "not-an-email" // COMPILE ERROR - constructor is hidden!
```


Runtime представление: тэги и layout

Суммы: хранятся как (тэг, данные)

- Тэг — целое число (обычно 1–4 байта)
- Данные — $\max(\text{size}(A), \text{size}(B))$ с padding
- **Стоимость case:** 1–2 сравнения, branch prediction, $O(1)$

Оптимизации:

- **Niche optimization** (Rust): `Option<&T>` = размер `&T` (тэг в null-указателе)
- **Unboxed types** (Haskell): inline данные, избегая указателей
- **Monomorphization** (Rust/C++): специализация под каждый тип

Вывод: ADT часто бесплатны или очень дешёвы в рантайме!

Чек-лист проектирования API

Перед реализацией задайте вопросы:

- ❶ Какие **состояния** может иметь объект? → Сумма
- ❷ Какие **поля** обязательны в каждом состоянии? → Произведение в варианте
- ❸ Могут ли состояния комбинироваться? Если нет — сумма, если да — произведение
- ❹ Есть ли **инварианты**? → Smart constructors, newtypes
- ❺ Есть ли **протокол** переходов? → Phantom types
- ❻ Нужна ли открытость для расширения? Если нет — sealed/closed

Правило: начинайте с самого строгого представления. Ослабляйте только при необходимости.

Антипаттерны: чего избегать

- **Строковая типизация**: `String` для `UserId`, `Email`, `JSON` → Решение: `newtypes`
- **Boolean blindness** («слепота булевых»): `fn send(encrypt: bool, compress: bool, retry: bool)` → Решение: `enum Config { Encrypted | Plain }`
- **Option-heavy records**: много `Option` полей → Решение: разбить на сумму состояний
- **Открытые иерархии без нужды** → Решение: `sealed traits/closed ADT`
- **Игнорирование exhaustiveness warnings** → Решение: всегда обрабатывайте все случаи

Цель: ошибки компиляции вместо рантайм-ошибок!

Ключевые принципы:

- ❶ **Сумма vs произведение**: суммы для взаимоисключения, произведения для обязательных полей
- ❷ **Make illegal states unrepresentable**: проектируйте типы так, чтобы невалидные комбинации были невыразимы
- ❸ **Phantom types**: кодируйте протоколы в типах (zero-cost)
- ❹ **Newtypes**: оборачивайте примитивы для семантики
- ❺ **Smart constructors**: валидируйте при создании
- ❻ **Runtime**: ADT дешёвы — тэги $O(1)$, niche optimization даёт zero overhead

Главный принцип: используйте систему типов для выражения инвариантов — получайте ошибки компиляции вместо рантайм-ошибок!

План лекции

- 1 Мотивация и основные понятия
- 2 Паттерн-матчинг, произведения и суммы
- 3 Алгебра типов
- 4 Рекурсивные ADT и полиномиальные функторы
- 5 Классы типов (ad-hoc полиморфизм)
- 6 Практические рекомендации
- 7 Практикум: QUIC протокол как ADT (quinn-proto)**
- 8 Заключение

Зачем QUIC для практики с типами? I

Проблема

Теория ADT и паттерн-матчинга абстрактна. Как применить к реальным системам с корректностью и безопасностью?

Решение: quinn-proto

Quinn-proto — детерминированная sans-I/O машина состояний для протокола QUIC (Rust).

- **Sans-I/O**: чистая логика без сетевого ввода-вывода — идеально для типового анализа
- **Реальный код**: живой проект с актуальными релизами (crates.io)
- **Богатые ADT**: enums для событий, structs для состояний, traits для политик
- **Security-critical**: ошибки в автомате ведут к уязвимостям (CVE-2024-45311)

Зачем QUIC для практики с типами? II

Цель практикума: построить упрощённую версию QUIC state machine, используя суммы/произведения, исчерпывающий `match` и классы типов для политик безопасности.

Ссылки: <https://github.com/quinn-rs/quinn>, <https://docs.rs/quinn-proto>

Архитектура quinn: sans-I/O подход I

Традиционный подход

- Протокол + I/O смешаны
- Сложно тестировать
- Недетерминированность
- Трудно формализовать

$\text{handle} : \text{Socket} \rightarrow \text{IO}()$

Sans-I/O (quinn-proto)

- Логика отделена от I/O
- Чистая функция переходов
- Детерминированность
- Типы выражают протокол

$\text{step} : \text{State} \times \text{Event} \rightarrow \text{State} \times [\text{Action}]$

Ключевая идея

Протокол — это чистая функция из $(\text{State}, \text{Event})$ в $(\text{State}', \text{Actions})$. Все I/O происходит снаружи.

Связь с ADT: State и Event — суммы типов, Action — сумма, переходы — `match`.

Реальные типы quinn-proto: обзор I

Из docs.rs/quinn-proto — примеры реальных ADT:

Произведения (structs)

- `Endpoint` — точка подключения
- `Connection` — состояние соединения
- `TransportConfig` — параметры
- `IdleTimeout` — таймауты
- `PathStats` — статистика пути

$\text{Connection} \cong \text{ConnId} \times \text{State} \times \text{Streams} \times \dots$

Суммы (enums)

- `Event` — внешние события
- `ConnectionEvent` — события соединения
- `StreamEvent` — события потоков
- `TransportError` — ошибки
- `ConnectError` — ошибки подключения

$\text{Event} \cong \text{Datagram Bytes} + \text{Timeout} + \text{AppClose} + \dots$

Traits (классы типов): `TokenStore`, `TimeSource`, `ConnectionIdGenerator` — абстракции для внешних зависимостей (как `Monoid/Functor`, но для домена).

Учебная модель: типы для практикума I

Упрощённая версия QUIC для понимания ADT и match. F*-псевдокод:

Произведения и суммы:

```
// ConnState (sum type)
type conn_state =
  | Initial of conn_id
  | Handshaking of bool // keysReady
  | Established of nat * nat // peerMaxStreams * ours
  | Draining
  | Closed of transport_error

// Event (sum type)
type event =
  | RxDatagram of datagram
  | TimerFired
  | AppOpenBi
```

Учебная модель: типы для практикума II

```
| AppClose  
| PeerClose of transport_error  
| KeysReady
```

Наблюдение: ConnState — сумма из 5 вариантов, Event — сумма из 6 вариантов.

Действия и функция перехода I

Действия (выход автомата):

```
// Action (sum type)
type action =
  | Tx of list datagram
  | SetTimer of float // seconds
  | AppNotify of event
  | Noop
```

Функция перехода (тотальная!):

Действия и функция перехода II

```
val step : conn_state -> event -> Tot (conn_state * list action)
let step state evt = match (state, evt) with
  // Initial x Event (6 branches)
  | (Initial odcid, RxDatagram dg) -> ...
  | (Initial odcid, TimerFired) -> (Closed Timeout, [])
  | (Initial _, AppClose) -> (Closed AppRequested, [])
  // ...
  // Handshaking x Event (6 branches)
  | (Handshaking false, KeysReady) -> (Handshaking true, [])
  | (Handshaking true, RxDatagram dg) ->
    (Established (100, 0), [AppNotify Connected])
  // ...
  // And so on for all 5 x 6 = 30 combinations
```

Ключ: функция *тотальна* — каждая пара $(State, Event)$ обработана. Exhaustiveness checking!

Применение изоморфизмов

Используем дистрибутивность $\sigma \times (\tau + \rho) \cong \sigma \times \tau + \sigma \times \rho$ для систематического покрытия.

$$\text{ConnState} \times \text{Event} \cong \text{Initial} \times \text{Event} + \text{Handshaking} \times \text{Event} + \dots$$

Матрица:

	RxDgram	Timer	AppOpenBi	AppClose	PeerClose	KeysReady
Initial	✓	✓	✓	✓	✓	✓
Handshaking	✓	✓	×	✓	✓	✓
Established	✓	✓	✓	✓	✓	×
Draining	×	✓	×	×	×	×
Closed	×	×	×	×	×	×

Алгебра типов: матрица покрытия II

✓ — осмысленная ветка, × — игнорируется (Noop). **Всего:** $5 \times 6 = 30$ веток.

Примеры игнорируемых переходов:

- Draining × AppOpenBi: соединение закрывается, новые потоки запрещены
- Closed × любое: соединение завершено, все события игнорируются
- Established × KeysReady: ключи уже установлены, событие невозможно

Чек-лист: при добавлении нового события/состояния обновляем матрицу.

Кардинальности: подсчёт вариантов I

Экстенсиональная интерпретация

$|\text{ConnState}| = 5$ (если упростить произведения в вариантах до 1).

$|\text{Event}| = 6$.

$|\text{Action}| = 4$ (без учёта списков).

Число возможных переходов:

$$|\text{ConnState} \times \text{Event}| = 5 \times 6 = 30$$

Число возможных результатов (упрощённо):

$$|\text{ConnState} \times \text{Action}| = 5 \times 4 = 20$$

(На практике Action — список, так что ∞ , но семантически 4 варианта.)

Изоморфизм для нового события

Добавили событие RetryReceived: новый $\text{Event}' = \text{Event} + 1$.

$$\text{ConnState} \times \text{Event}' \cong (\text{ConnState} \times \text{Event}) + (\text{ConnState} \times 1)$$

Нужно добавить 5 веток (по одной на состояние).

Policy как typeclass: абстракция безопасности I

Идея: политики лимитов, TTL, валидации токенов — это *классы типов* (traits в Rust).

```
// Policy type class (dictionary)
type policy (p:Type) = {
  allowed_open: p -> conn_state -> bool;
  allowed_use: p -> conn_state -> event -> bool;
  ttl_expired: p -> float -> bool
  // Monotonicity law:
  // If p' is stricter than p
  // (forall s. allowed_use p' s e ==> allowed_use p s e),
  // then the set of allowed actions does not grow.
}
```

Реализации:

Policy как typeclass: абстракция безопасности II

```
// Permissive policy
type permissive_policy = | Permissive

let permissive_policy_dict : policy permissive_policy = {
  allowed_open = (fun _ _ -> true);
  allowed_use = (fun _ _ _ -> true);
  ttl_expired = (fun _ _ -> false)
}

// Strict policy
type strict_policy = {
  max_streams: nat;
  timeout: float
}
```

Policy как typeclass: абстракция безопасности III

```
let strict_policy_dict : policy strict_policy = {  
  allowed_open = (fun p s -> match s with  
    | Established (_, ours) -> ours < p.max_streams  
    | _ -> false);  
  allowed_use = (fun _ _ _ -> true); // can be extended  
  ttl_expired = (fun p elapsed -> elapsed > p.timeout)  
}
```

Инварианты: связь с Лекцией 3 I

Вспомним результаты из Лекции 3 ($\text{Safety} = \text{Progress} + \text{Preservation}$). Применим к нашему автомату:

I1: Канонические формы

Значение типа `ConnState` есть один из конструкторов: `Initial`, `Handshaking`, ..., `Closed`.

Аналог: каждое значение типа-суммы — это конкретный конструктор (из Л4, слайд про канонические формы).

I2: Прогресс

Для любых замкнутых $s : \text{ConnState}$ и $e : \text{Event}$ функция $\text{step}(s, e)$ либо редуцируется к значению, либо делает β -шаг (разбор case).

Практика: нет «застывших» состояний — всегда есть переход.

I3: Сохранение

β -шаги для case/проекций сохраняют тип результата.

Практика: тип $(ConnState, [Action])$ всегда корректен после шага.

Safety: исчерпывающий match + сохранение типов $\Rightarrow$ нет runtime errors.

Связь с реальной безопасностью: CVE-2024-45311

Реальный инцидент в quinn-proto 0.11

Вызов `retry()` на невалидированном соединении приводит к панике при последующем `refuse()/ignore()` на валидированном соединении (при дублирующем `initial` пакете).

Причина: CWE-670 (Always-Incorrect Control Flow) — некорректная последовательность переходов между состояниями `unvalidated` → `validated` → `refused`.

Как типы помогают предотвратить:

- 1 **Phantom types:** кодировать статус валидации в типе — `Connection<Unvalidated>` vs `Connection<Validated>`
- 2 **Линейные типы:** `retry()` потребляет `Incoming<Unvalidated>`, исключая повторное использование
- 3 **Exhaustive match:** компилятор требует обработки всех (`State`, `ValidationStatus`, `Event`)
- 4 **State machine verification:** формальная проверка достижимости состояний

Концептуальный пример (упрощённо):

- До: `fn retry(&mut self)` и `fn refuse(&mut self)` оба изменяют `self.state` независимо → паника

Детальный разбор transition function I

Конкретная реализация перехода Initial → Handshaking:

```
val step : conn_state -> event -> Tot (conn_state * list action)
let step state evt = match (state, evt) with
| (Initial odcid, RxDatagram dg) ->
    (match parse_packet dg with
    | ClientHello params ->
        let server_hello = build_server_hello odcid params in
        let new_state = Handshaking false in
        let actions = [Tx [server_hello]; SetTimer 3.0] in
        (new_state, actions)
    | _ -> (Initial odcid, [Noop]))

| (Handshaking false, KeysReady) ->
    (Handshaking true, [AppNotify KeysEstablished])

| (Handshaking true, RxDatagram dg) ->
```

Детальный разбор transition function II

```
(match parse_packet dg with  
  | ClientFinished ->  
    (Established (100, 0), [AppNotify Connected])  
  | _ -> (Handshaking true, [Noop]))  
  
| _ -> ... // other branches
```

Ключевое: каждый переход явно обрабатывает пакеты — нет «скрытой логики».

Stream management: вложенные суммы и произведения I

Проблема: QUIC поддерживает множество потоков (streams) с независимым состоянием.

Типы для потоков:

```
type stream_id = nat

type direction = | Send | Recv

type stream_state =
  | StreamOpen of nat * nat // bytesReceived * bytesSent
  | StreamHalfClosed of direction
  | StreamClosed

// Connection state now includes stream map
type conn_state =
  | Initial of conn_id
  | Handshaking of bool
```

Stream management: вложенные суммы и произведения II

```
| Established of established_state
| Draining
| Closed of transport_error

and established_state = {
  peer_max_streams: nat;
  our_streams: FStar.Map.t stream_id stream_state;
  stream_credits: nat
}
```

Наблюдение: Established теперь произведение с *коллекцией* сумм (Map StreamId StreamState).

Flow control: инварианты через типы I

Инварианты flow control

- **I-FC1:** $\forall s \in \text{streams. bytesSent}(s) \leq \text{sendWindow}(s)$
- **I-FC2:** суммарные отправленные байты $\leq$ connection-level window
- **I-FC3:** credits не могут быть отрицательными

Выражение в типах (refined types):

$\text{StreamOpen} : \{\text{bytesSent} : \mathbb{N} \mid \text{bytesSent} \leq \text{sendWindow}\} \times \dots$

В Rust (без зависимых типов):

- Smart constructors: `fn send(stream: &mut Stream, data: &[u8]) -> Result<(), FlowControlError>`
- Runtime проверки с ранним возвратом ошибки

Flow control: инварианты через типы II

- Но типы *документируют* инвариант через сигнатуру

Идея: даже без зависимых типов, `Result<(), FlowControlError>` лучше, чем `panic!` или `undefined behavior`.

Cryptographic handshake: phantom types для фаз I

Проблема: на разных фазах handshake доступны разные операции.

Решение: phantom types (как в слайде про `Connection<Authenticated>`).

```
// Phantom type tags for phases
type initial_phase
type handshake_phase
type application_data_phase

// Crypto with phantom parameter
type crypto (phase:Type) =
  | CryptoInitial : keys initial_phase -> crypto initial_phase
  | CryptoHS      : keys handshake_phase -> crypto handshake_phase
  | CryptoApp     : keys application_data_phase -> crypto application_data_phase

// Type-safe operations per phase
val encrypt_initial : crypto initial_phase -> packet ->
```

Cryptographic handshake: phantom types для фаз II

```
                Tot encrypted_packet
let encrypt_initial (CryptoInitial keys) pkt = ...

// Cannot call encrypt_app on Initial phase - type error!
val encrypt_app : crypto application_data_phase -> packet ->
                Tot encrypted_packet
let encrypt_app (CryptoApp keys) pkt = ...
```

Гарантия: невозможно случайно использовать ключи из неправильной фазы — компилятор не даст!

Retry tokens: address validation через типы I

Проблема DDoS: клиент может подделать адрес; сервер должен проверить, что клиент владеет адресом.

Механизм Retry token:

```
type token = {  
    token_value: bytes;  
    token_issued: timestamp;  
    client_addr: address  
}  
  
type conn_state =  
    | Initial of conn_id  
    | AwaitingRetry of conn_id * token  
    | ValidatedInitial of conn_id * address  
    | Handshaking of bool  
    | Established of established_state
```

Retry tokens: address validation через типы II

| ...

```
let step (state:conn_state) (evt:event)
  : Tot (conn_state * list action) =
  match (state, evt) with
  | (Initial odcid, RxDatagram dg) ->
    if needs_validation (client_addr dg) then
      let token = issue_token (client_addr dg) in
      (AwaitingRetry (odcid, token), [Tx [RetryPacket token]])
    else
      (ValidatedInitial (odcid, client_addr dg), [...])

  | (AwaitingRetry (odcid, issued_token), RxDatagram dg) ->
    (match extract_token dg with
     | Some received_token ->
       if received_token = issued_token then
```

Retry tokens: address validation через типы III

```
        (ValidatedInitial (odcid, client_addr dg), [...])  
      else (AwaitingRetry (odcid, issued_token), [Noop])  
    | None -> (AwaitingRetry (odcid, issued_token), [Noop]))  
  
    | _ -> ...
```

Инвариант: в состоянии Established мы всегда прошли через ValidatedInitial.

Congestion control как typeclass I

Идея: алгоритм congestion control (Reno, Cubic, BBR) — это *политика*, абстрагированная через typeclass.

```
// Congestion Control type class (dictionary)
type congestion_control (cc:Type) = {
  on_packet_sent: cc -> nat -> Tot cc;
  on_packet_acked: cc -> nat -> float -> Tot cc;  // bytes, rtt
  on_packet_lost: cc -> nat -> Tot cc;
  get_congestion_window: cc -> Tot nat
  // Laws:
  // 1. Window monotonically decreases on loss
  // 2. Window increases on successful ack (in congestion avoidance)
  // 3. get_congestion_window always returns non-negative value
}

type cubic = {
```

Congestion control как typeclass II

```
    cwnd: nat;
    ssthresh: nat
    // ... other fields
}

type bbr = {
    bottleneck_bw: nat;
    min_rtt: float
    // ... other fields
}

let cubic_cc : congestion_control cubic = {
    on_packet_sent = (fun cc bytes -> cc);
    on_packet_acked = (fun cc bytes rtt ->
        if cc.cwnd < cc.ssthresh then // slow start
            { cc with cwnd = cc.cwnd + bytes }
    )
}
```

Congestion control как typeclass III

```
    else
      ... // cubic increase formula
  );
  on_packet_lost = (fun cc bytes ->
    { cc with ssthresh = cc.cwnd / 2; cwnd = cc.ssthresh });
  get_congestion_window = (fun cc -> cc.cwnd)
}
```

Преимущество: можно менять алгоритм СС без изменения Connection logic — подстановка разных инстансов.

Реальный код quinn-proto: структура I

Ключевые типы (quinn-proto/src/):

- `struct Connection` — большое произведение: внутреннее состояние, потоки, packet spaces (Initial/Handshake/Application), таймеры, статистика
- `enum Event` — сумма событий: `HandshakeDataReady`, `Connected`, `ConnectionLost`, `Stream(StreamEvent)`, etc.
- `struct Endpoint` — управляет конфигурацией и распределяет датаграммы по `Connection`
- `trait` интерфейсы: `crypto::Session`, `ConnectionIdGenerator`, congestion control

Ключевые методы `Connection`:

- `handle_event()` — обработка входящих датаграмм
- `poll_transmit()` — получение данных для отправки
- `poll_timeout()` — управление таймерами

Реальный код quinn-proto: структура II

Внутри — обширный `match` по состояниям и событиям, ровно как в нашей модели!

Вывод: наша упрощённая модель отражает реальную архитектуру production-системы.

Паттерны обработки: стиль реального кода I

```
// Inspired by quinn-proto (simplified pseudocode)
fn on_ack_received(&mut self, space: SpaceId, ack: frame::Ack)
    -> Result<(), TransportError>
{
    let space = &mut self.spaces[space];
    for range in ack.iter() {
        for packet_num in range {
            if let Some(sent) = space.sent_packets.remove(&packet_num) {
                // Update congestion control (trait call)
                self.congestion.on_packet_acked(sent.size, sent.rtt);

                // Update stream state via pattern matching
                for frame in sent.frames {
                    match frame {
                        Frame::Stream(s) => self.streams.on_data_acked(s),
                        Frame::Crypto(c) => self.crypto.on_acked(c),
```

Паттерны обработки: стиль реального кода II

```
}  
  Ok()  
}  
  }  
}  
  }  
}  
  _ => {}  
}
```

Паттерны: match по enum вариантам, trait-based abstractions, Result для ошибок.

- 1 **Sans-I/O**: отделение логики от I/O — ключ к типовому анализу протоколов.
- 2 **ADT для состояний**: ConnState — сумма, каждый вариант — произведение полей.
- 3 **Тотальность**: функция step покрывает все (*State*, *Event*) — exhaustiveness checking как «прогресс».
- 4 **Алгебра типов**: дистрибутивность $\times$ и $+$ даёт матрицу покрытия и чек-лист.
- 5 **Typeclasses для политик**: абстракция безопасности, законы (монотонность).
- 6 **Инварианты из ЛЗ**: канонические формы, прогресс, сохранение — safety = no stuck.
- 7 **Реальная безопасность**: CVE показывают, что типы — не роскошь, а необходимость.

Главный takeaway: хорошо спроектированный ADT делает целый класс ошибок невозможным на уровне компиляции!

План лекции

- 1 Мотивация и основные понятия
- 2 Паттерн-матчинг, произведения и суммы
- 3 Алгебра типов
- 4 Рекурсивные ADT и полиномиальные функторы
- 5 Классы типов (ad-hoc полиморфизм)
- 6 Практические рекомендации
- 7 Практикум: QUIC протокол как ADT (quinn-proto)
- 8 **Заключение**

- Расширили STLC произведениями ($\times$), суммами ($+$), 1 и 0.
- Для каждого типа задали Formation, Introduction, Elimination и Computation.
- Сформулировали β/η -равенства и операционную семантику.
- Рассмотрели алгебраическую интерпретацию типов и изоморфизмы.
- Показали соответствие паттерн-матчинга чистым элиминаторам.
- Обсудили важность покрытия вариантов для безопасности типов.
- **Изучили практики проектирования API с ADT**: сумма vs произведение, phantom types, newtypes, smart constructors.

Дальше: полиморфизм, зависимые типы, индуктивные семейства.